Review of ImageNet Classification with Deep Convolutional Neural Networks

Abhishu Panthi
Pulchowk Campus, IOE
Tribhuvan University
080bct009.abhishu@pcampus.edu.np

Ishan Gautam
Pulchowk Campus, IOE
Tribhuvan University
080bct034.ishan@pcampus.edu.np

Kushal K.C.
Pulchowk Campus, IOE
Tribhuvan University
080bct041.kushal@pcampus.edu.np

***Abstract*—In this paper, the contributions of Krizhevsky, Sutskever, and Hinton on ImageNet classification via deep convolutional neural networks, which scored top-5 test error rate of 15.3% on ILSVRC-2012. We will analyze the architecture of the Activations for faster convergence, and multi-GPU parallelization, dropout regularization on fully-connected layers, and extensive data augmentation using random cropping, horizontal flipping, and PCA-based color transformation perturbation. The training procedure relied on stochastic gradient descent with momentum, weight decay, and adaptive learning rate scheduling. We examine particular design decisions, including Local Response Normalization, Overlapping Pooling, Weight Initialization strategies. This paper will cover learned feature representations, demonstrating GPU specialization and semantic abstraction in higher layers. We place the AlexNet architecture in the context of subsequent developments, analyzing its shallow depth, large 11×11 kernels, and LRN and their evolution into batch normalization, smaller filters and ResNet architectures.**

***Index Terms*—Deep Learning, Convolutional Neural Networks, ImageNet, AlexNet, Computer Vision, Image Classification, ReLU, LRN, Dropout, GPU Computing**

I. Introduction

To effectively learn about thousands of objects from millions of images, we need a model with a large learning capacity. Convolutional neural networks (CNNs) constitute one such class of models [5], [6]. Their capacity can be controlled by varying their depth and breadth, compared to standard feedforward neural networks with similarly sized layers, CNNs have much fewer connections and parameters. Alexnet is a deep convolutional neural network to classify the 1.2 million high-resolution images into the 1000 different classes. The neural network, which has 60 million parameters and 650,000 neurons consists of five convolutional layers some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax [1].

A. Historical Context

Prior to 2012, computer vision systems primarily relied on hand-crafted features combined with traditional machine learning algorithms particularly, methods such as SIFT (Scale-Invariant Feature Transform), Histograms of Oriented Gradients (HOG) to extract visual patterns [2], [3]. The limitation of these approaches was the inability to scale with increasing complexity of visual data. These methods were powerful but were limited by lack of adaptability.

B. Dataset

ImageNet dataset of over 15 million labeled high-resolution images belonging to roughly 22,000 categories [4]. ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) uses a subset of ImageNet with roughly 1000 images in each of 1000 categories [7]. In all, there are roughly 1.2 million training images, 50,000 validation images, and 150,000 testing images. Each image was down-sampled to a fixed resolution of 256×256.

II. Network Architecture

A. Overall Design

AlexNet consists of eight learned layers: five convolutional layers followed by three fully-connected layers [1]. The network processes 224×224×3 RGB images and output of the last fully-connected layer is fed to a 1000-way softmax which produces distribution over 1000 class labels, with approximately 60 million parameters.

B. Convolutional Layers

The initial layer uses 96 kernels sized 11×11×3, which convolve with the input image using a stride of 4 pixels. These large stride helps reduce the output spatial volume size significantly. This layer is responsible for capturing low-level features such as oriented edges and color blobs, which are then passed through ReLU activation, Local Response Normalization (LRN) and max-pooling. The subsequent layers of the network refine these initial features into increasingly abstract representations. The second convolution layer uses 256 kernels of size 5×5×48. Use of 48 feature maps each corresponds to separate filtered outputs from previous layer. Layers 3, 4 are then connected directly without pooling and normalization layers. Third layer includes 384 kernels of size 3×3×256, fourth mirrors the third layer's configuration without altering layer's spatial dimensions using 384 kernels of size 3×3×192. Final kernel uses 256 kernels of size 3×3×192 followed by max-pooling layer which reduces dimensionality as well as rotational and positional invariance to the features being learned.

C. Fully-Connected Layers

The first and second fully connected layer's is a dense layer of 4096 neurons, it takes flattened output from preceding

convolution layers and projects onto high-dimensional space to learn non-linear combination of these features. Final fully connected layer has as many neurons as there are classes (1000), and utilizes a softmax function to derive the classification probabilities. The softmax also forms basis for cross-entropy loss during training. The convolutional layers comprise most of the units and connections, but the fully connected layers are responsible for most of the weights.

D. Multi-GPU Architecture

AlexNet utilizes parallelization across two GPUs to overcome the hardware limitations. Specifically, the network was split across two NVIDIA GTX 580 GPUs, each with only 3 GB of memory. The network distributes kernels across both GPUs, with inter-GPU communication occurring only at specific layers. For instance, layer 3 kernels receive inputs from all kernel maps in layer 2, where as layer 4 kernels only receive inputs form layer 3 on the same GPU.

III. AlexNet Components

A. ReLU Nonlinearity

AlexNet introduced the use of Rectified Linear Units (ReLUs) as activation functions in deep convolutional neural networks. The ReLU function is defined as:

$$f(x) = \max(0, x) \tag{1}$$

In contrast to traditional saturating nonlinearities such as $\tanh(x)$ or sigmoid functions $\sigma(x) = (1 + e^{-x})^{-1}$, ReLUs allow networks converge faster by addressing vanishing gradient. The gradient of the ReLU function is either 0 for negative inputs or 1 for positive inputs, which speeds up gradient descent. Since it outputs zero for half of its input domain, it inherently produces sparse data representations. The authors demonstrated that networks with ReLUs train several times faster than equivalent networks with saturating activation functions, reaching 25% training error on CIFAR-10 approximately six times faster than networks using $\tanh$ units. [1]

B. Local Response Normalization

Although ReLUs do not require input normalization to prevent saturation, AlexNet uses local response normalization (LRN) to aid generalization. For a neuron with activity $a^i_{x,y}$ at position (x, y) using kernel i, the normalized activity $b^i_{x,y}$ is computed as:

$$b^i_{x,y} = a^i_{x,y} \Bigg/ \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2 \right)^{\beta} \tag{2}$$

where the summation runs over n adjacent kernel maps at the same spatial position, N is the total number of kernels in the layer, and k, α, β, and n are hyper-parameters.

LRN implements a form of lateral inhibition, a biological process where activated neurons supress the activity of neighboring neurons in same layer, creating competition among neuron outputs computed using different kernels. This inhibition contributes to activity regulation by preventing any single feature map from overwhelming the response of the network by penalizing larger activations that lack support from their surroundings. By focusing on patterns that stand out relative to their neighbors, LRN functions similarly to contrast normalization. Incorporating LRN reduced the top-1 and top-5 error rates by 1.4% and 1.2%, respectively [1].

C. Overlapping Pooling

Traditionally, the neighborhoods summarized by adjacent pooling units do not overlap [8]. However, AlexNet applies overlapping pooling with $s = 2$ and $z = 3$, causing adjacent pooling windows to overlap by one pixel. This increases the density of the pooled feature maps by approximately 33% in linear dimension compared to non-overlapping pooling with $s = z = 3$. Overlapping pooling increases computational cost in the pooling layer by a factor of approximately $(z/s)^2 = 2.25$ compared to non-overlapping pooling, since each pixel (except borders) is involved in multiple pooling operations. However, increasing the density of feature mapping while making the model slightly more resistant to overfitting. Reducing top-1 and top-5 error rates by 0.4% and 0.3% respectively [1].

IV. Regularization Techniques

A. Data Augmentation

The size of network made overfitting a significant problem, even with 1.2 million labeled training examples and 60 million parameters, so AlexNet addresses this through data augmentation:

1) Image Translations and Reflections: The network extracts random 224×224 patches from 256×256 images during training, along with their horizontal reflections. This increased the training set size by a factor of 2,048 [1]. At test time, the network averages predictions over ten patches (four corners, center, and their horizontal reflections) for more stable results.

2) PCA-based Color Augmentation: For each training image, the method adds multiples of principal components to RGB values:

$$[p_1, p_2, p_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T \tag{3}$$

where p_i and λ_i are the i-th eigenvector and eigenvalue of the RGB covariance matrix, and α_i are random variables drawn from a Gaussian distribution with mean zero and standard deviation 0.1. This captures the invariance of object identity to changes in illumination intensity and color, reducing top-1 error rate by over 1% [1]. Also the transformed images are CPU generated while the GPU is training on the previous batch of images. So these data augmentation schemes are, in effect, computationally free.

B. Dropout

Dropout regularization randomly sets the output of hidden neurons to zero with probability 0.5 during training. This prevents complex co-adaptations of neurons, as each neuron

must learn robust features useful in conjunction with many random subsets of other neurons [9].

At test time, all neurons are active but their outputs are multiplied by 0.5, approximating the geometric mean of predictions from the exponentially many dropout networks. Applied to the first two fully-connected layers, dropout roughly doubles the iterations required for convergence but substantially reduces overfitting.

V. Training Methodology and Optimization

A. Optimization

The network is trained using stochastic gradient descent (SGD) with the following hyperparameters:

- Batch size: 128 examples
- Momentum: 0.9
- Weight decay: 0.0005
- Initial learning rate: 0.01

The weight update rule follows:

$$v_{i+1} := 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w}\Big|_{w_i} \right\rangle_{D_i} \tag{4}$$

$$w_{i+1} := w_i + v_{i+1} \tag{5}$$

where i is the iteration index, v is the momentum variable, ϵ is the learning rate, and $\langle \partial L/\partial w|_{w_i} \rangle_{D_i}$ represents the average gradient over batch D_i. The batch size choice was balance between computational efficiency and accuracy of estimating errors, while larger batch size provide a clearer signal for each update by reducing noise in gradient calculations, they also require more computational resources. High level of momentum speeds up convergence towards the loss function's minimum, particularly useful when dealing with small but consistent gradients. Weight decay acts as a regularization term that penalizes large weights by adding a portion of weight values to the loss function, ensuring that model does not rely too heavily on small number of high-weight features. Learning rate is decreased by factor of 10 at predetermined points during training also known as *step decay*, these adjustments are made typically when the validation error rate stops decreasing significantly. Reducing it helps refine weight adjustments, promoting better convergence.

B. Initialization

Weights are initialized from a zero-mean Gaussian distribution with standard deviation 0.01, narrow standard deviation prevents any single neuron from initially overwhelming the output. Neuron biases in the second, fourth, fifth convolutional layers and fully-connected layers, were initialized to 1.0 to provide ReLUs with positive inputs initially. Remaining layer biases were initialized to 0 [1].

VI. Results and Performance

A. ILSVRC-2010 Results

On the ILSVRC-2010 test set, AlexNet achieved top-1 and top-5 error rates of 37.5% and 17.0% respectively [1]. This represented a substantial improvement over previous methods:

- Sparse coding approach: 47.1% top-1, 28.2% top-5
- SIFT + Fisher Vectors: 45.7% top-1, 25.7% top-5

B. ILSVRC-2012 Competition

In the ILSVRC-2012 competition:

- Single CNN: 18.2% top-5 error
- Ensemble of 5 CNNs: 16.4% top-5 error
- CNN pre-trained on Fall 2011 release: 16.6% top-5 error
- Ensemble of 7 CNNs: 15.3% top-5 error (winning entry)

The second-place entry achieved 26.2% top-5 error [1].

C. Ablation Studies

Removing any single convolutional layer resulted in approximately 2% degradation in top-1 performance, despite each convolutional layer containing less than 1% of the model's parameters [1], so the depth really is important for achieving results.

VII. Analysis and Discussion

A. Learned Features

Examining the convolutional kernels learned by the model showed interpretable patterns. The first layer learns frequency-selective and orientation-selective filters resembling Gabor filters, along with color blob detectors. Interestingly, the two GPUs exhibit specialization: GPU-1 learns largely color-agnostic features while GPU-2 learns color-specific features and is independent of any particular random weight initialization.

Feature similarity analysis using Euclidean distance in the 4,096-dimensional space of the last hidden layer shows that the network learns semantically meaningful representations. The model successfully abstracts high-level concepts, as evidenced by retrieved nearest neighbors often share semantic similarity despite substantial pixel-level differences.

B. Evolution

While AlexNet marked a monumental leap in performance, its architectural and methodological choices have been rapidly refined by subsequent research. These limitations are not failures but rather initial design points that were optimized as the field matured.

1) **Depth:** Eight-layer architecture was the practical limit of trainable depth in 2012, but it constrained the network's representational capacity for hierarchical feature learning. AlexNet is relatively shallow compared to subsequent architectures like VGG-Net (19 layers) [10], GoogLeNet (22 layers) [11], and ResNet (up to 152 layers) [12].
2) **Large filters:** The 11×11 kernels in the first layer are computationally expensive, kernel contains 121 parameters per channel, for first layer with 96 filters and 3 input channels, this resulted in 34,848 parameters for the first layer alone. Evolution to smaller filters followed that n stacked 3×3 convolutions could yield effective receptive field of $(2n+1)(2n+1)$ while requiring only $9n$ parameters versus $(2n+1)^2$ parameters for

a single single large convolutional filter [10]. VGGNet formalized this by employing homogeneous stacks of 3×3 convolutions, while Inception architectures combined multiple filter sizes in parallel branches for multi-scale feature extraction within a single layer [11].

3) **Local Response Normalization:** Local Response Normalization (LRN) was introduced as a form of lateral inhibition but addressed neither covariate shift nor gradient flow stabilization and relied on empirically chosen (k,α,β,n). Batch Normalization (BN) provides a solution to internal covariate shift by changing distribution of layer inputs during training [13]. BN normalizes activations within each mini-batch as follows:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \quad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2, \tag{6}$$

$$\hat{y}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma\hat{y}_i + \beta, \tag{7}$$

where γ and β are learnable parameters that restore the representational capacity of the network. By stabilizing activation distributions, it allows for higher learning rates and accelerates convergence. What that means is the inputs are linearly transformed to have zero mean and unit variance. The network converges faster and shows better regularization during training, which has an impact on the overall accuracy.

4) **Weight initialization:** Gaussian initialization with $W \sim \mathcal{N}(0, 0.01^2)$, ignores signal propagation in deep networks. This causes exponential growth or decay of activation variances across layers. Newer approach such as Xavier initialization preserves forward-pass variance [14].

$$\mathrm{Var}(W) = \frac{2}{n_{\mathrm{in}} + n_{\mathrm{out}}} \tag{8}$$

VIII. Conclusion

AlexNet established a foundational architectural paradigm that catalyzed the modern era of deep learning. It proved that deep convolutional neural network is capable of achieving record-breaking results on a highly challenging dataset using purely supervised learning. Author's note that the network's performance degrades even if a single convolutional layer is removed, indicating that depth is highly important. The paper's technical contributions-ReLU activation, dropout regularization, GPU acceleration, and effective data augmentation-have become standard. While subsequent architectures have improved upon AlexNet's performance through increased depth, normalization techniques, and connectivity patterns, the core remain relevant.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25*, 2012, pp. 1097–1105.

[2] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," Int. J. Comput. Vis., vol. 60, no. 2, pp. 91–110, Nov. 2004.

[3] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. 2005 IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR'05), San Diego, CA, USA, Jun. 2005, vol. 1, pp. 886–893.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2009, pp. 248–255.

[5] K. Jarrett, K. Kavukcuoglu, M. A. Ranzato, and Y. LeCun, "What is the best multi-stage architecture for object recognition?," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2009, pp. 2146–2153.

[6] H. Lee, R. Grosse, R. Ranganath, and A. Y. Ng, "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations," in Proc. Int. Conf. Mach. Learn. (ICML), 2009, pp. 609–616.

[7] J. Deng, A. Berg, S. Satheesh, H. Su, A. Khosla, and L. Fei-Fei, "ILSVRC-2012," 2012. [Online]. Available: http://www.image-net.org/challenges/LSVRC/2012/

[8] Y. LeCun, K. Kavukcuoglu, and C. Farabet, "Convolutional networks and applications in vision," in Proc. IEEE Int. Symp. Circuits Syst. (ISCAS), 2010, pp. 253–256.

[9] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," arXiv:1207.0580, 2012.

[10] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv:1409.1556, 2014.

[11] C. Szegedy et al., "Going deeper with convolutions," arXiv:1409.4842, 2014.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Las Vegas, NV, USA, Jun. 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[13] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Mach. Learn. (ICML), Lille, France, Jul. 2015, pp. 448–456.

[14] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in Proc. 13th Int. Conf. Artif. Intell. Stat. (AISTATS), Sardinia, Italy, May 2010, pp. 249–256.